Documento de Projeto Detalhado de Software
Sistema Runner

Descrição de Projeto de Software baseada na IEEE 1016-2009

Sistema	Sistema Runner
Documento	Projeto Detalhado de Software / Software Design Description
Norma de referência	IEEE 1016-2009 — Software Design Description
Contexto	Disciplina de Implementação e Integração de Software — Engenharia de Software
Versão do documento	1.0
Data	28/04/2026
Fontes de entrada	especificacao.md; design.md; plano-revisitado-v2.md; sprint-1-tasks.md; diagramas C4 anexados

Histórico de revisão

Versão	Data	Descrição	Responsável
1.0	28/04/2026	Elaboração inicial do documento de projeto detalhado de software.	Bárbara Nogueira Carvalho da Silveira

Sumário informativo

1. Introdução
2. Visão geral do projeto de software
3. Stakeholders, preocupações e critérios de projeto
4. Pontos de vista e visões de projeto
5. Projeto de decomposição em módulos e pacotes
6. Projeto detalhado dos componentes Go
7. Projeto detalhado do assinador.jar em Java
8. Projeto das interfaces externas e internas
9. Projeto de dados, arquivos locais e estruturas de mensagem
10. Projeto comportamental e fluxos de execução
11. Projeto de tratamento de erros, logs e mensagens
12. Projeto de build, testes, CI/CD e distribuição
13. Rastreabilidade entre requisitos e elementos de projeto
14. Decisões de projeto, justificativas, riscos e pendências
15. Referências e apêndices

1. Introdução

1.1 Finalidade

Este documento apresenta o projeto detalhado de software do Sistema Runner. Seu objetivo é transformar a especificação de requisitos e a arquitetura já definidas em uma descrição operacional suficientemente precisa para orientar a implementação, os testes, a manutenção e a evolução do sistema.

A estrutura foi organizada de forma compatível com a IEEE 1016-2009, que trata de descrições de projeto de software por meio de visões, pontos de vista, elementos de projeto, atributos, justificativas e rastreabilidade. O documento não reproduz a norma, mas aplica seus conceitos ao contexto acadêmico do Sistema Runner.

1.2 Escopo do documento

O documento cobre o projeto detalhado dos CLIs assinatura e simulador, dos pacotes internos em Go, do assinador.jar em Java, das interfaces de integração local e HTTP, dos mecanismos de provisionamento de JDK/JRE, do gerenciamento de processos, dos arquivos locais em ~/.hubsaude/, dos workflows de build e release e da rastreabilidade entre requisitos e componentes de projeto.

O escopo deste projeto detalhado considera as quatro sprints previstas: fundação e entrega contínua; assinatura digital simulada em modo local; modo servidor com HTTP e PKCS#11; e gerenciamento completo do Simulador do HubSaúde.

1.3 Produto de software descrito

O Sistema Runner é uma solução de linha de comandos que facilita a execução de aplicações Java sem exigir que o usuário conheça detalhes de instalação, configuração, parâmetros da JVM ou localização de arquivos .jar. Ele é composto por binários Go multiplataforma, por uma aplicação Java de assinatura simulada e por mecanismos de integração com o Simulador do HubSaúde.

1.4 Definições, siglas e abreviações

Termo	Definição
CLI	Command Line Interface; interface de linha de comandos utilizada pelo usuário no terminal.
Runner	Sistema responsável por facilitar a execução de aplicações Java por meio de comandos simples.
assinatura	CLI principal usado para operações de criação e validação de assinatura simulada.
simulador	CLI usado para iniciar, parar e consultar o status do Simulador do HubSaúde.
assinador.jar	Aplicação Java responsável por validar parâmetros e simular operações de assinatura e validação.
PKCS#11	Interface padrão para comunicação com tokens criptográficos e smart cards.
JDK/JRE	Ambiente Java usado para executar ou desenvolver aplicações Java.
SemVer	Versionamento semântico no formato MAJOR.MINOR.PATCH, por exemplo v0.1.0.
Cosign	Ferramenta do ecossistema Sigstore usada para assinar e verificar artefatos distribuídos.
OIDC	OpenID Connect; mecanismo de identidade usado na assinatura automatizada de artefatos.

2. Visão geral do projeto de software

2.1 Problema a ser resolvido

O problema central é permitir que integradores da Plataforma HubSaúde executem aplicações Java por meio de comandos simples, sem lidar diretamente com

instalação do Java, paths, comandos java -jar, portas, processos em background, downloads manuais e verificação de artefatos.

O projeto também deve simular fluxos de assinatura digital e validação de assinatura, mantendo foco na validação rigorosa dos parâmetros, no tratamento de erros e na integração entre componentes heterogêneos.

2.2 Objetivos de projeto

- Separar claramente a interface de usuário em linha de comandos da implementação Java responsável pela simulação de assinatura.
- Permitir execução local do assinador.jar por meio de java -jar e execução em modo servidor via HTTP.
- Gerenciar automaticamente JDK/JRE, arquivos .jar, cache local, processos e artefatos baixados.
- Criar uma estrutura modular que facilite testes, evolução incremental e manutenção.
- Garantir portabilidade para Windows, Linux e macOS em arquitetura amd64.
- Preparar a distribuição dos binários por GitHub Releases com checksums e assinatura de artefatos.
- Manter rastreabilidade entre requisitos, decisões de projeto, componentes e testes.

2.3 Restrições e premissas técnicas

Identificador	Restrição ou premissa	Impacto no projeto
RP-01	Os CLIs serão desenvolvidos em Go 1.25.	A decomposição de módulos Go deve favorecer cross-compilation e binários autossuficientes.
RP-02	O assinador.jar será desenvolvido em Java 21.	O projeto deve prever detecção/provisionamento de JDK/JRE 21 e empacotamento Maven.
RP-03	O sistema deve suportar Windows, Linux e macOS amd64.	Rotinas de path, extensão .exe, permissões, encerramento de processos e extração de

		arquivos devem ser portáveis.
RP-04	A assinatura digital será simulada.	O projeto prioriza validação de parâmetros e resposta controlada, não implementação criptográfica real.
RP-05	A integração com dispositivo criptográfico é via PKCS#11.	O assinador.jar deve isolar esse acesso em adaptador próprio para permitir teste com SoftHSM2 ou equivalente.
RP-06	Artefatos devem ser distribuídos via GitHub Releases.	O projeto inclui workflows, versionamento SemVer, checksums e assinatura dos arquivos.
RP-07	Informações locais devem ser armazenadas em ~/.hubsaude/.	O projeto inclui cache, registro de processos, JDK/JRE gerenciado e metadados de versão nesse diretório.

2.4 Visão C4 de contexto e contêineres

As figuras a seguir foram usadas como base para o detalhamento. O Diagrama de Contexto mostra o usuário, o Sistema Runner, o Dispositivo de Assinatura Digital e o Simulador do HubSaúde. O Diagrama de Contêineres detalha os CLIs, o assinador.jar e os sistemas externos.

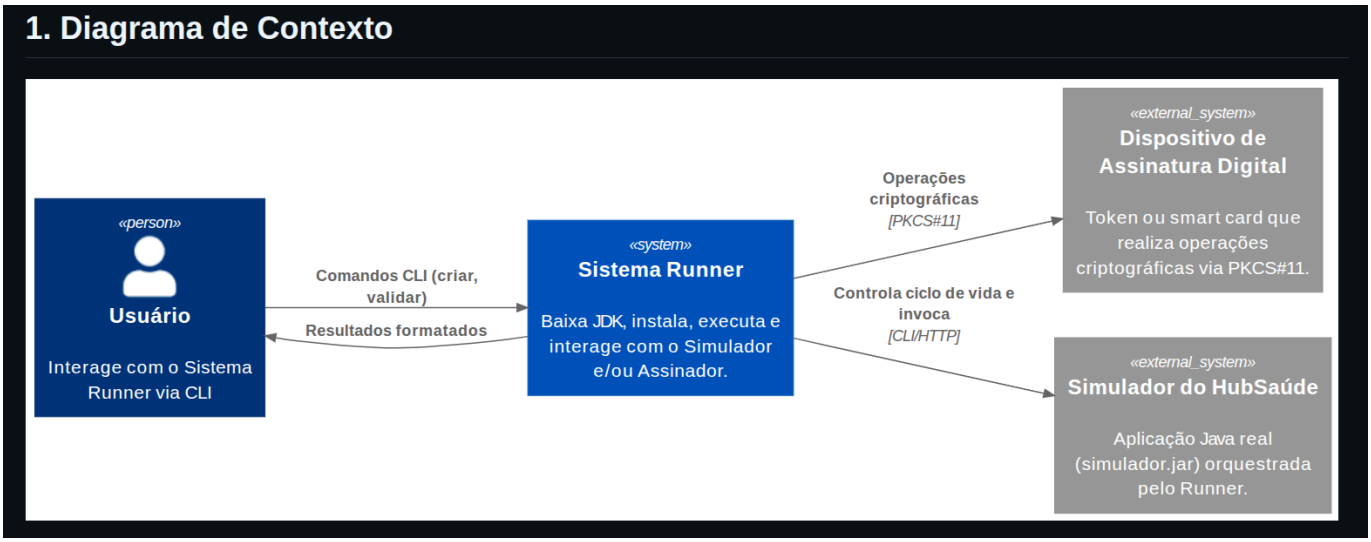

Figura 1 — Diagrama de Contexto do Sistema Runner.

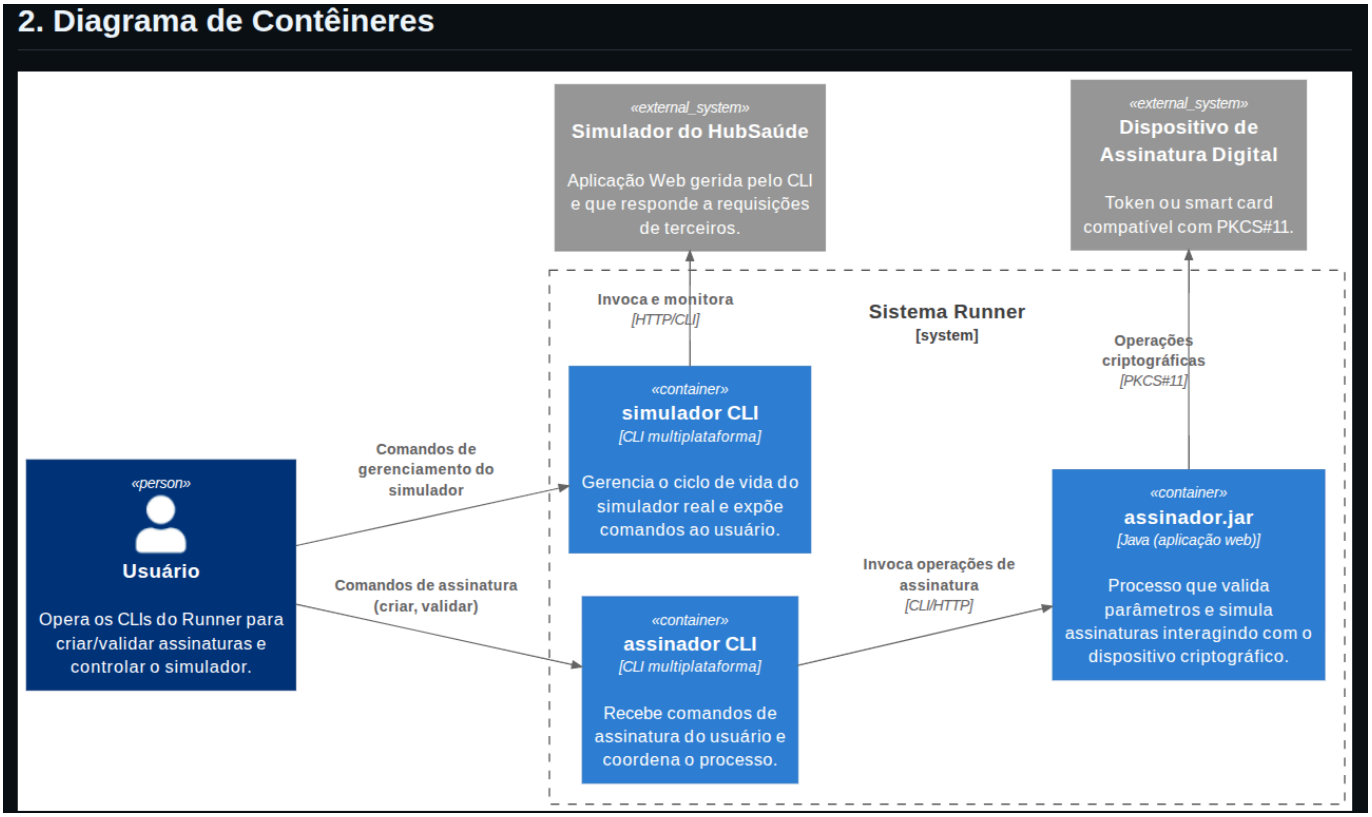

Figura 2 — Diagrama de Contêineres do Sistema Runner.

3. Stakeholders, preocupações e critérios de projeto

3.1 Stakeholders

Stakeholder	Interesse no projeto detalhado
Usuário final técnico	Executar assinatura, validação e simulador por comandos simples, com mensagens claras e sem configurar Java manualmente.
Desenvolvedor Go	Entender a organização dos pacotes, as interfaces internas, os comandos CLI e a lógica de integração.
Desenvolvedor Java	Implementar assinador.jar, validações, endpoints HTTP, serviços de assinatura simulada e integração PKCS#11.
Mantenedor/DevOps	Manter workflows de CI/CD, releases, checksums, assinatura de artefatos e compatibilidade multiplataforma.

Professor/Avaliador	Avaliar se o projeto cumpre requisitos, arquitetura, rastreabilidade, qualidade, integração e critérios de aceitação.
Integrador HubSaúde	Compreender como o Runner facilita a execução de aplicações Java e o gerenciamento do simulador.

3.2 Preocupações de projeto

Código	Preocupação	Resposta de projeto
C-01	Portabilidade	Uso de Go para CLIs, cross-compilation, abstrações de filesystem/processos e testes em três runners.
C-02	Integração Go-Java	Pacote invoker isola a chamada local por java -jar e a chamada HTTP ao assinador.jar.
C-03	Usabilidade de CLI	Comandos com Cobra, --help, mensagens de erro orientativas e saída estruturada.
C-04	Manutenibilidade	Separação em pacotes internos, interfaces pequenas, DTOs, validadores e adaptadores.
C-05	Testabilidade	Interfaces para invocadores, filesystem, downloader, JDK resolver e serviços Java simulados.
C-06	Segurança de distribuição	Checksums SHA256, Cosign, assinatura automática e publicação controlada em Releases.
C-07	Gerenciamento de processos	Registro de PID/porta, health check, encerramento limpo e remoção de registros obsoletos.

C-08	Simulação confiável	Validação rigorosa antes da resposta simulada e separação entre regra de validação e retorno fake.

3.3 Critérios de qualidade do projeto

Critério	Aplicação no projeto detalhado
Clareza	Cada componente possui responsabilidade explícita, interfaces definidas e dependências conhecidas.
Baixo acoplamento	Os CLIs dependem de interfaces internas; a execução Java fica isolada em invocadores.
Alta coesão	Pacotes agrupam responsabilidades específicas: CLI, invocação, JDK, releases, processos e saída.
Extensibilidade	Novos comandos, novos endpoints ou novos provedores de download podem ser adicionados sem alterar todo o sistema.
Observabilidade básica	Mensagens e logs devem indicar comando executado, modo de invocação, porta, caminho do jar e causa de falhas.
Resiliência a erro operacional	JDK ausente, jar não encontrado, porta ocupada, servidor inativo e checksum inválido devem gerar mensagens tratáveis.

4. Pontos de vista e visões de projeto

Para organizar o projeto detalhado conforme o espírito da IEEE 1016-2009, este documento usa pontos de vista de projeto. Cada ponto de vista define um tipo de preocupação e produz uma visão correspondente do sistema.

Ponto de vista	Preocupações atendidas	Visões geradas
VP-01 Decomposição modular	Organização em pacotes, componentes, responsabilidades e	Visão de pacotes Go, módulos Java e componentes principais.

	unidades de implementação.	
VP-02 Interface	Contratos entre usuário, CLIs, assinador.jar, simulador.jar, GitHub Releases e dispositivo PKCS#11.	Visão de comandos, endpoints HTTP, argumentos e DTOs.
VP-03 Dados e estado	Arquivos locais, cache, registro de processos, manifesto de releases, respostas JSON.	Visão de modelos de dados e persistência local.
VP-04 Comportamento	Fluxos de assinatura, validação, start/stop/status, fallback e provisionamento.	Visão de cenários, algoritmos e sequências.
VP-05 Construção e implantação	Build, testes, releases, artefatos multiplataforma, checksums e assinatura.	Visão de CI/CD e distribuição.
VP-06 Qualidade e validação	Testabilidade, tratamento de erros, segurança de artefatos e critérios de aceite.	Visão de testes, rastreabilidade e riscos.

5. Projeto de decomposição em módulos e pacotes

5.1 Estrutura física do repositório

A estrutura física proposta concentra os dois binários Go em cmd/, os pacotes reutilizáveis em internal/ e o projeto Java em assinador/. Essa organização permite CI unificada e separa entrypoints de bibliotecas internas.

```
runner/
├── cmd/
│   ├── assinatura/
│   │   ├── main.go
│   │   └── version_test.go
│   └── simulador/
│       └── main.go
├── internal/
│   ├── cli/
│   ├── invoker/
```

```
│    ├── jdk/
│    ├── release/
│    ├── process/
│    ├── output/
│    └── config/
├── assinador/
│    ├── pom.xml
│    └── src/main/java/...
├── .github/workflows/
│    ├── build.yml
│    └── release.yml
├── go.mod
└── go.sum
```

5.2 Mapa de componentes de alto nível

Componente	Tecnologia	Responsabilidade principal	Interage com
cmd/assinatura	Go	Inicializar o CLI de assinatura, registrar comandos e expor a versão do binário.	internal/cli, internal/invoker, usuário
cmd/simulador	Go	Inicializar o CLI do simulador e registrar comandos start, stop, status e version.	internal/cli, internal/release, internal/process
internal/cli	Go	Definir comandos, flags, parsing, validações iniciais e mensagens de ajuda.	Cobra, invoker, output
internal/invoker	Go	Executar assinador.jar localmente ou via HTTP e padronizar respostas.	jdk, process, net/http, os/exec

internal/jdk	Go	Detectar Java no sistema ou provisionar JDK/JRE gerenciado.	filesystem, release/downloader
internal/release	Go	Baixar JDK/JRE, simulador.jar e artefatos, validando versão e checksum.	GitHub Releases, release.json
internal/process	Go	Registrar, consultar e encerrar processos gerenciados.	~/.hubsaude, OS process API
assinador.jar	Java 21	Validar parâmetros e simular assinatura/validação em modo CLI ou HTTP.	CLI assinatura, PKCS#11
Simulador HubSaúde	Java/Web	Responder a /api/info e /shutdown, sendo gerenciado pelo CLI simulador.	CLI simulador, terceiros
Workflows CI/CD	GitHub Actions	Testar, compilar, publicar artefatos e assinar releases.	GitHub, Go, Cosign

5.3 Dependências entre pacotes Go

```
cmd/assinatura   ──▶ internal/cli ──▶ internal/invoker ──▶
internal/jdk
           │                        │                    │
└──▶ internal/release
           │                        │                    └──▶
internal/process
        │                  └──▶ internal/output
        └──▶ version injection via -ldflags

cmd/simulador    ──▶ internal/cli ──▶ internal/release ──▶
```

```
internal/jdk
        │                    └──▶ internal/process
        └──▶ internal/output
```

Regra de dependência: pacotes em internal/ não devem depender de cmd/. Os entrypoints cmd/ apenas compõem os comandos e injetam configurações. Isso evita acoplamento circular e facilita testes unitários dos pacotes internos.

6. Projeto detalhado dos componentes Go

6.1 Componente cmd/assinatura

Aspecto	Descrição
Responsabilidade	Ser o ponto de entrada do binário assinatura. Deve criar o root command, registrar subcomandos e executar a aplicação.
Arquivo principal	cmd/assinatura/main.go.
Variável de versão	var version = "dev"; deve ser variável, não constante, para permitir injeção por -ldflags.
Comandos previstos	version, sign, validate, start, stop, status.
Dependências	internal/cli para construção dos comandos; internal/output para exibição; internal/invoker para operações.
Testes	Teste de aceitação com go run . version verificando a saída dev no ambiente local.

```
// Exemplo de organização esperada, não código final obrigatório
func main() {
                                                root :=
cli.NewAssinaturaRootCommand(cli.AssinaturaOptions{
        Version: version,
        InvokerFactory: invoker.NewFactory(),
                Output:  output.NewTerminalWriter(os.Stdout,
os.Stderr),
    })
```

```
    if err := root.Execute(); err != nil {
        os.Exit(cli.ExitCode(err))
    }
}
```

6.2 Componente cmd/simulador

Aspecto	Descrição
Responsabilidade	Ser o ponto de entrada do binário simulador, dedicado ao gerenciamento do Simulador do HubSaúde.
Arquivo principal	cmd/simulador/main.go.
Comandos previstos	version, start, stop, status.
Estado inicial	Na Sprint 1, pode ser um stub que imprime mensagem de construção; nas sprints posteriores recebe comandos completos.
Dependências	internal/cli, internal/process, internal/release, internal/jdk e internal/output.

6.3 Pacote internal/cli

O pacote internal/cli concentra a definição dos comandos e flags. Ele não deve conter lógica pesada de negócio; sua função é traduzir entrada textual do usuário em chamadas para serviços internos.

Item	Projeto detalhado
Tipos principais	AssinaturaOptions, SimuladorOptions, CommandFactory, ValidationError.
Funções principais	NewAssinaturaRootCommand, NewSimuladorRootCommand, bindSignFlags, bindValidateFlags, bindProcessFlags.
Validações iniciais	Presença de parâmetros obrigatórios, formato básico de porta, arquivos informados e flags incompatíveis.
Regras	Não deve acessar diretamente os.Exit dentro das funções testáveis; deve retornar erro para o main decidir exit code.
Saídas	Deve usar internal/output para imprimir sucesso, erro e ajuda customizada.

6.4 Pacote internal/invoker

O pacote internal/invoker isola a comunicação com o assinador.jar. Ele fornece uma interface única para o restante do CLI, independentemente de a execução ser local ou HTTP.

Elemento	Descrição
SignatureInvoker	Interface com métodos Sign(ctx, request), Validate(ctx, request), Health(ctx), Start(ctx, options), Stop(ctx, options).
LocalInvoker	Executa java -jar assinador.jar com argumentos mapeados a partir do request.
HTTPInvoker	Envia POST /sign e POST /validate para uma instância HTTP do assinador.jar.
InvokerFactory	Escolhe HTTP quando existe instância saudável; usa local quando o usuário passa --local ou quando servidor não está disponível.
CommandBuilder	Monta argumentos de linha de comando de forma segura, sem concatenação textual vulnerável.
ResponseParser	Converte JSON ou saída estruturada do assinador.jar em objetos de domínio do CLI.

```
type SignatureInvoker interface {
     Sign(ctx context.Context, req SignRequest) (SignResult, error)
        Validate(ctx context.Context, req ValidateRequest) (ValidateResult, error)
    Health(ctx context.Context, port int) error
          Start(ctx context.Context, opts ServerOptions) (ProcessInfo, error)
    Stop(ctx context.Context, opts StopOptions) error
}
```

6.5 Pacote internal/jdk

O pacote internal/jdk é responsável por localizar um Java compatível. A decisão de projeto é permitir execução com Java do sistema quando compatível, mas também manter um JDK/JRE gerenciado dentro de ~/.hubsaude/.

Classe/Tipo	Responsabilidade
JDKResolver	Orquestra a busca por Java compatível e retorna o caminho executável java.
SystemJavaLocator	Procura java no PATH e verifica versão.
ManagedJavaLocator	Procura Java baixado em ~/.hubsaude/jdk ou ~/.hubsaude/jre.
AdoptiumDownloader	Baixa distribuição adequada à plataforma quando ausente.
ArchiveExtractor	Extrai arquivo baixado com tratamento por sistema operacional.
JavaVersion	Representa e compara versão mínima requerida, preferencialmente 21.

6.6 Pacote internal/release

Elemento	Responsabilidade
ReleaseManifest	Representar release.json contendo URLs e versões de jar e JRE/JDK por plataforma.
ManifestClient	Baixar release.json a partir de URL fixa ou alternativa.
ArtifactCache	Determinar se o artefato já está disponível localmente e se sua versão corresponde à mais recente.
Downloader	Fazer download com timeout, progresso opcional e tratamento de falhas de rede.
ChecksumVerifier	Validar SHA256 dos arquivos baixados e impedir uso de artefato corrompido.
CosignVerifier	Ponto de extensão para verificação de assinatura de artefatos quando aplicável.

6.7 Pacote internal/process

O pacote internal/process controla processos externos iniciados pelo Runner. Ele deve lidar com diferenças entre sistemas operacionais e evitar processos duplicados.

Elemento	Responsabilidade
ProcessRegistry	Ler e escrever registros de processos gerenciados em ~/.hubsaude/processes/.
ProcessInfo	Representar nome, PID, porta, comando, caminho do jar, data de início e status conhecido.
PortChecker	Verificar disponibilidade de porta antes de iniciar servidor ou simulador.
HealthChecker	Confirmar que processo registrado responde em endpoint esperado.
ProcessTerminator	Encerrar processo por PID ou por endpoint HTTP de shutdown, conforme o componente.

6.8 Pacote internal/output

Elemento	Responsabilidade
TerminalWriter	Centralizar escrita em stdout/stderr.
ResultFormatter	Formatar SignResult e ValidateResult em texto legível.
ErrorFormatter	Transformar erros internos em mensagens orientativas ao usuário.
JSONFormatter	Ponto de extensão para saída --json, útil em automação.

7. Projeto detalhado do assinador.jar em Java

7.1 Organização proposta do projeto Java

```
assinador/
├── pom.xml
└── src/
    ├── main/java/br/gov/go/saude/hubsaude/assinador/
    │   ├── Application.java
    │   ├── cli/
```

```
    │   │   └── AssinadorCli.java
    │   ├── http/
    │   │   ├── SignatureController.java
    │   │   └── GlobalExceptionHandler.java
    │   ├── service/
    │   │   ├── SignatureService.java
    │   │   └── FakeSignatureService.java
    │   ├── validation/
    │   │   ├── SignRequestValidator.java
    │   │   └── ValidateRequestValidator.java
    │   ├── crypto/
    │   │   ├── Pkcs11Adapter.java
    │   │   └── CryptoDeviceStatus.java
    │   └── dto/
    │       ├── SignRequest.java
    │       ├── SignResponse.java
    │       ├── ValidateRequest.java
    │       ├── ValidateResponse.java
    │       └── ErrorResponse.java
    └── test/java/...
```

7.2 Componentes Java

Componente	Responsabilidade	Observações de projeto
Application	Ponto de entrada do jar.	Deve decidir modo CLI ou HTTP com base nos argumentos, por exemplo --server.
AssinadorCli	Receber argumentos em modo local e chamar serviços internos.	Deve retornar stdout estruturado e exit code apropriado.
SignatureController	Expor endpoints HTTP POST /sign, POST /validate e GET /health.	Reutiliza os mesmos validadores e serviço usados no modo CLI.
SignatureService	Contrato de serviço para assinatura e validação.	Permite trocar implementação fake por real no futuro.

FakeSignatureService	Gerar respostas simuladas pré-construídas.	Não executa assinatura criptográfica real.
SignRequestValidator	Validar parâmetros de criação de assinatura.	Não deve permitir processamento quando parâmetros obrigatórios estão ausentes ou inválidos.
ValidateRequestValidator	Validar parâmetros de validação de assinatura.	Deve gerar mensagens que indiquem campo e motivo do erro.
Pkcs11Adapter	Isolar comunicação com dispositivo criptográfico.	Permite teste com SoftHSM2 ou simulador equivalente.
GlobalExceptionHandler	Padronizar erros HTTP.	Retorna ErrorResponse com código, mensagem e detalhes.

7.3 Contratos internos Java

```
public interface SignatureService {
    SignResponse sign(SignRequest request);
    ValidateResponse validate(ValidateRequest request);
}

public interface RequestValidator<T> {
    void validate(T request) throws ValidationException;
}

public interface CryptoDeviceAdapter {
    CryptoDeviceStatus status();
    Optional<String> selectedCertificate(String alias);
}
```

7.4 Regras de validação do assinador.jar

Regra	Descrição	Comportamento em erro
VAL-01	Campos obrigatórios de criação de assinatura devem estar presentes.	Retornar erro indicando o campo ausente.

VAL-02	Identificador do documento deve ter formato válido e não vazio.	Retornar erro de parâmetro inválido.
VAL-03	Conteúdo ou referência do documento deve existir e ser legível quando aplicável.	Retornar erro de arquivo/recurso não encontrado.
VAL-04	Algoritmo informado deve pertencer à lista aceita pela simulação.	Retornar erro com lista de valores aceitos.
VAL-05	Parâmetros de validação devem conter assinatura e documento/referência associados.	Retornar erro de entrada incompleta.
VAL-06	Quando PKCS#11 for solicitado, configuração/dispositivo deve ser verificável.	Retornar erro de dispositivo indisponível, sem encerrar a aplicação abruptamente.

8. Projeto das interfaces externas e internas

8.1 Interface de usuário: CLI assinatura

Comando	Finalidade	Parâmetros principais	Resultado esperado
assinatura version	Exibir a versão atual do CLI.	Nenhum.	Texto com a versão, por exemplo dev ou v0.1.0.
assinatura sign	Solicitar criação de assinatura simulada.	--input, --document-id, --signer-id, --cert-alias, --algorithm, --local, --port, --json.	Assinatura simulada ou erro de validação.
assinatura validate	Solicitar validação de assinatura simulada.	--input, --signature, --document-id, --local, --port, --json.	Resultado válido/inválido e justificativa.

assinatura start	Iniciar assinador.jar em modo servidor.	--port, --timeout, --jar.	Processo iniciado, PID e porta registrados.
assinatura status	Consultar instância do assinador.jar.	--port.	Status, PID, porta e saúde HTTP.
assinatura stop	Encerrar instância do assinador.jar.	--port.	Servidor encerrado e registro atualizado.

8.2 Interface de usuário: CLI simulador

Comando	Finalidade	Parâmetros principais	Resultado esperado
simulador version	Exibir versão do CLI simulador.	Nenhum.	Texto com a versão.
simulador start	Iniciar o simulador.jar.	--port, --source, --timeout.	Simulador iniciado, porta e PID registrados.
simulador status	Consultar status do simulador.	--port.	Informações obtidas de /api/info ou indicação de inativo.
simulador stop	Parar o simulador.	--port.	Chamada ao endpoint /shutdown e limpeza de registro.

8.3 Interface HTTP do assinador.jar

Endpoint	Método	Entrada	Saída	Observações
/health	GET	Nenhuma.	Status do serviço.	Usado por HealthChecker e pelo CLI para reaproveitar instância ativa.
/sign	POST	SignRequest em JSON.	SignResponse ou ErrorResponse.	Reutiliza validação e serviço do modo CLI.

/validate	POST	ValidateRequest em JSON.	ValidateResponse ou ErrorResponse.	Simula validação de assinatura.
/shutdown	POST	Opcional.	Confirmação de encerramento.	Pode ser usado para encerramento controlado do assinador, se habilitado.

8.4 Interface HTTP do Simulador do HubSaúde

Endpoint	Método	Uso pelo Runner	Comportamento esperado
/api/info	GET	Consultar status do simulador.	Retornar informações básicas da aplicação em execução.
/shutdown	POST	Solicitar parada controlada do simulador.	Encerrar o processo ou sinalizar encerramento.

8.5 Interface com dispositivo criptográfico

A comunicação com token ou smart card deve ser isolada no Pkcs11Adapter. O restante do assinador.jar não deve depender diretamente de classes SunPKCS11 espalhadas pelo código. Isso preserva testabilidade e permite que a simulação funcione mesmo sem dispositivo real.

Operação	Entrada	Saída	Falha prevista
status	Configuração PKCS#11 opcional.	Disponível, indisponível ou não configurado.	Biblioteca nativa ausente, token removido ou PIN não fornecido.
selectedCertificate	Alias ou identificador do certificado.	Informação simulada ou certificado encontrado.	Certificado não encontrado ou dispositivo inacessível.

9. Projeto de dados, arquivos locais e estruturas de mensagem

9.1 Diretório local ~/.hubsaude/

Caminho	Conteúdo	Uso
~/.hubsaude/jdk/ ou ~/.hubsaude/jre/	Distribuição Java gerenciada.	Executar assinador.jar e simulador.jar quando Java não estiver no PATH.
~/.hubsaude/artifacts/	Jars baixados, checksums e metadados.	Cache local de assinador.jar, simulador.jar e arquivos relacionados.
~/.hubsaude/processes/	Arquivos JSON com PID, porta e status conhecido.	Reutilizar ou encerrar processos já iniciados.
~/.hubsaude/logs/	Logs de execução local, downloads e erros.	Diagnóstico de problemas e auditoria simples.
~/.hubsaude/config.json	Configuração global opcional.	Portas padrão, origem de releases, preferências de saída.

9.2 Modelo SignRequest

Campo	Tipo	Obrigatório	Descrição
documentId	string	Sim	Identificador lógico do documento a ser assinado.
inputPath	string	Condicional	Caminho do arquivo de entrada quando a operação usa arquivo.
payload	string/base64	Condicional	Conteúdo informado diretamente, quando não houver arquivo.
signerId	string	Sim	Identificador do solicitante ou signatário.
certificateAlias	string	Não	Alias do certificado no dispositivo ou simulação.

algorithm	string	Não	Algoritmo solicitado; valor padrão definido pelo assinador.jar.
pkcs11	object	Não	Configuração opcional do dispositivo criptográfico.

9.3 Modelo SignResponse

Campo	Tipo	Descrição
signatureId	string	Identificador da assinatura simulada.
status	string	Resultado da simulação, por exemplo SIGNED.
signedAt	datetime	Data/hora em que a resposta simulada foi gerada.
algorithm	string	Algoritmo registrado na resposta.
signatureValue	string	Valor textual/base64 pré-construído ou gerado pela simulação.
certificateSubject	string	Sujeito do certificado simulado ou selecionado.
warnings	array	Avisos não impeditivos, quando existirem.

9.4 Modelo ValidateRequest e ValidateResponse

Modelo	Campo	Tipo	Descrição
ValidateRequest	documentId	string	Identificador do documento associado à assinatura.
ValidateRequest	inputPath/payload	string	Documento ou conteúdo usado na validação simulada.
ValidateRequest	signatureValue	string	Assinatura a validar.

ValidateRequest	certificateInfo	object/string	Informação de certificado, quando aplicável.
ValidateResponse	valid	boolean	Indica se a assinatura foi considerada válida pela regra simulada.
ValidateResponse	status	string	VALID, INVALID ou ERROR.
ValidateResponse	reason	string	Justificativa textual do resultado.
ValidateResponse	validatedAt	datetime	Data/hora da validação simulada.

9.5 Modelo ErrorResponse

Campo	Tipo	Descrição
code	string	Código estável do erro, por exemplo PARAM_REQUIRED, JDK_NOT_FOUND ou CHECKSUM_FAILED.
message	string	Mensagem clara para o usuário.
details	array/object	Detalhes técnicos ou campos inválidos.
suggestion	string	Orientação de correção quando possível.
traceId	string	Identificador opcional para correlacionar logs.

9.6 Modelo ProcessInfo

```
{
  "name": "assinador",
  "pid": 12345,
  "port": 8080,
  "mode": "http-server",
  "jarPath": "/home/user/.hubsaude/artifacts/assinador.jar",
  "startedAt": "2026-04-28T12:00:00-03:00",
```

```
  "healthUrl": "http://127.0.0.1:8080/health"
}
```

9.7 Modelo ReleaseManifest

```
{
  "jar": {
                                                          "url":
"https://github.com/kyriosdata/assinador/releases/latest/dow
nload/assinador.jar",
    "version": "1.2.0",
    "sha256": "..."
  },
  "jre": {
                                                     "windows_x64":
"https://api.adoptium.net/v3/binary/latest/21/ga/windows/x64
/jre/hotspot/normal/eclipse",
                                                       "linux_x64":
"https://api.adoptium.net/v3/binary/latest/21/ga/linux/x64/j
re/hotspot/normal/eclipse",
                                                         "mac_x64":
"https://api.adoptium.net/v3/binary/latest/21/ga/mac/x64/jre
/hotspot/normal/eclipse"
  }
}
```

10. Projeto comportamental e fluxos de execução

10.1 Fluxo de assinatura em modo local

1. Usuário executa assinatura sign com parâmetros de entrada.
2. internal/cli valida presença básica dos parâmetros e constrói SignRequest.
3. InvokerFactory seleciona LocalInvoker quando --local é informado ou quando não há servidor saudável.
4. JDKResolver localiza Java compatível no PATH ou em ~/.hubsaude/.
5. LocalInvoker monta comando java -jar assinador.jar sign ... sem concatenação insegura de string.
6. assinador.jar valida parâmetros de forma rigorosa.

7. FakeSignatureService retorna SignResponse simulado quando os parâmetros são válidos.
8. LocalInvoker captura stdout/stderr e exit code.
9. ResultFormatter exibe resposta legível ou JSON, conforme flags.

10.2 Fluxo de assinatura em modo HTTP

10. Usuário executa assinatura sign sem forçar --local.
11. ProcessRegistry verifica se existe processo assinador registrado.
12. HealthChecker chama /health na porta registrada.
13. Se o servidor estiver saudável, HTTPInvoker envia POST /sign.
14. Se não houver servidor saudável, o CLI pode iniciar servidor ou fazer fallback local, conforme política configurada.
15. assinador.jar valida o JSON recebido e chama SignatureService.
16. HTTPInvoker interpreta SignResponse ou ErrorResponse.
17. Output formata o resultado para terminal.

10.3 Fluxo de validação

O fluxo de validação segue a mesma estrutura do fluxo de assinatura, trocando SignRequest por ValidateRequest e usando o método validate do serviço. O resultado deve informar claramente se a assinatura simulada foi considerada válida ou inválida e por qual motivo.

10.4 Fluxo de start do assinador.jar

18. Usuário executa assinatura start ou um comando que exige servidor ativo.
19. PortChecker verifica se a porta solicitada está disponível.
20. JDKResolver localiza Java compatível.
21. ArtifactResolver localiza assinador.jar ou baixa versão requerida.
22. ProcessManager inicia java -jar assinador.jar --server --port <porta>.
23. HealthChecker aguarda o endpoint /health responder dentro do timeout.
24. ProcessRegistry registra PID, porta, jarPath e startedAt.
25. CLI informa que o servidor foi iniciado.

10.5 Fluxo de stop do assinador.jar

26. Usuário executa assinatura stop --port <porta>.
27. ProcessRegistry busca processo registrado para a porta.
28. Se houver endpoint de shutdown, o CLI solicita encerramento controlado.

29. Se o processo continuar ativo após timeout, ProcessTerminator aplica encerramento por PID conforme o sistema operacional.
30. O registro local é removido ou marcado como inativo.
31. CLI exibe confirmação ou orientação de correção.

10.6 Fluxo de start/status/stop do simulador

Operação	Passos principais
simulador start	Verificar porta 8443 ou porta informada; localizar/baixar simulador.jar; localizar JRE/JDK; iniciar processo; aguardar /api/info; registrar PID e porta.
simulador status	Ler ProcessRegistry; chamar /api/info; exibir ativo/inativo, porta, PID, versão e caminho do jar.
simulador stop	Chamar /shutdown; aguardar término; remover registro; informar resultado ao usuário.

10.7 Algoritmo de provisionamento Java

```
resolveJava(requiredVersion = 21):
    if java no PATH existe e versão >= requiredVersion:
        return caminho_do_java_do_sistema
        if  java  gerenciado  existe  em  ~/.hubsaude/jdk  ou
~/.hubsaude/jre e versão >= requiredVersion:
        return caminho_do_java_gerenciado
      manifest  =  baixar  release.json  ou  usar  configuração
padrão
    url = selecionar URL por GOOS/GOARCH
    arquivo = baixar distribuição Java
    verificar checksum quando disponível
    extrair em ~/.hubsaude/jdk ou ~/.hubsaude/jre
    return caminho_do_java_gerenciado
```

10.8 Algoritmo de download de artefatos

```
resolveArtifact(name, versionPolicy):
    manifest = obter release.json
```

```
    target = selecionar artifact por nome e plataforma
    if existe em cache e versão/checksum conferem:
        return caminho_local
    baixar arquivo para caminho temporário
    verificar SHA256
    mover atomicamente para ~/.hubsaude/artifacts/
    registrar metadados de versão
    return caminho_local
```

11. Projeto de tratamento de erros, logs e mensagens

11.1 Categorias de erro

Categoria	Exemplos	Tratamento
Erro de parâmetro	Campo obrigatório ausente, porta inválida, arquivo não informado.	Mensagem direta indicando o parâmetro e exemplo de uso.
Erro de ambiente	Java ausente, versão incompatível, sem permissão de execução.	Orientar provisionamento automático ou ação manual.
Erro de artefato	Jar não encontrado, checksum inválido, download incompleto.	Não executar artefato; limpar temporário e sugerir nova tentativa.
Erro de rede	Falha ao baixar release.json, timeout HTTP, endpoint indisponível.	Informar URL/porta e permitir fallback quando possível.
Erro de processo	Porta ocupada, PID inexistente, processo zumbi.	Atualizar registro local e orientar status/stop.
Erro do assinador.jar	Validação Java rejeitada, dispositivo PKCS#11 indisponível.	Propagar ErrorResponse de forma legível.

11.2 Convenção de códigos de erro

Código	Significado	Mensagem sugerida
PARAM_REQUIRED	Parâmetro obrigatório ausente.	Informe o parâmetro --input. Use assinatura sign --help para exemplos.

PARAM_INVALID	Parâmetro com formato inválido.	O parâmetro --port deve ser um número entre 1 e 65535.
JDK_NOT_FOUND	Java compatível não encontrado.	Não foi encontrado Java 21. O Runner tentará baixar uma versão compatível.
JAR_NOT_FOUND	Arquivo .jar requerido não encontrado.	O assinador.jar não foi localizado no caminho informado ou no cache local.
PORT_IN_USE	Porta indisponível.	A porta solicitada já está em uso. Informe outra porta com --port.
HTTP_UNAVAILABLE	Servidor não responde.	A instância registrada não respondeu ao health check.
CHECKSUM_FAILED	Checksum diferente do esperado.	O artefato baixado pode estar corrompido e não será executado.
PKCS11_UNAVAILABLE	Dispositivo criptográfico indisponível.	Verifique se o token/smart card está conectado e configurado.

11.3 Logs

Logs devem ser suficientes para diagnóstico sem expor dados sensíveis. Por padrão, o CLI deve exibir mensagens simples. Detalhes técnicos podem ser gravados em ~/.hubsaude/logs/ ou exibidos com flag --verbose.

- Não registrar conteúdo integral de documentos assinados ou assinaturas sensíveis.
- Registrar caminho do jar, porta, modo de invocação, versão do Java e código de erro.
- Em modo --verbose, registrar detalhes de download e health check.
- Em erros de rede, registrar URL de origem e status HTTP, quando houver.

12. Projeto de build, testes, CI/CD e distribuição

12.1 Build local

```
go build ./...
go vet ./...
go test ./...

# Build manual dos binários principais
go build -o dist/assinatura ./cmd/assinatura
go build -o dist/simulador ./cmd/simulador
```

12.2 Testes previstos

Tipo de teste	Objetivo	Exemplos
Unitário Go	Testar validações, formatadores, resolução de paths e parsers.	Porta inválida, argumentos obrigatórios, ReleaseManifest.
Unitário Java	Testar validadores e FakeSignatureService.	SignRequest válido, campo ausente, algoritmo inválido.
Integração local	Validar CLI Go chamando assinador.jar por java -jar.	assinatura sign --local retorna SignResponse.
Integração HTTP	Validar CLI Go chamando endpoints /sign e /validate.	Servidor ativo, fallback quando health falha.
Processo	Validar start, status e stop em portas controladas.	Registro de PID e remoção após stop.
Download/cache	Validar download, checksum e reuso de cache.	Artefato já baixado não é baixado novamente.
Aceitação	Validar critérios de usuário.	assinatura version imprime dev localmente; releases geram artefatos.

12.3 Workflow build.yml

Job	Runner	Responsabilidade
test	ubuntu-latest, windows-latest, macos-latest	Executar go vet ./... e go test ./... em cada plataforma.

build	ubuntu-latest	Fazer cross-compilation dos binários para windows/amd64, linux/amd64 e darwin/amd64.
upload-artifacts	ubuntu-latest	Publicar artefatos de workflow para cada plataforma.

12.4 Workflow release.yml

Etapa	Responsabilidade
Trigger	Executar quando uma tag v* for criada, por exemplo v0.1.0.
test	Reexecutar go vet e go test nas três plataformas antes do release.
build	Compilar binários com versão injetada por -ldflags "-X main.version=<tag>".
checksums	Gerar checksums.txt com SHA256 de todos os binários.
cosign	Assinar artefatos e gerar arquivos .sig e .pem, quando habilitado no pipeline.
publish	Criar GitHub Release e anexar binários, checksums e assinaturas.

12.5 Convenção de nomes de artefatos

Plataforma	Nome esperado
Linux amd64	assinatura-v0.1.0-linux-amd64; simulador-v0.1.0-linux-amd64
Windows amd64	assinatura-v0.1.0-windows-amd64.exe; simulador-v0.1.0-windows-amd64.exe
macOS amd64	assinatura-v0.1.0-darwin-amd64; simulador-v0.1.0-darwin-amd64

13. Rastreabilidade entre requisitos e elementos de projeto

Requisito	Elementos de projeto	Testes associados
US-01 Invocar assinador.jar via CLI	cmd/assinatura, internal/cli, internal/invoker,	Testes de parsing, integração CLI→jar,

	LocalInvoker, HTTPInvoker, ResultFormatter.	integração CLI→HTTP, teste de version.
US-02 Simular assinatura digital com validação	assinador.jar, SignatureService, FakeSignatureService, validadores, DTOs, ErrorResponse, Pkcs11Adapter.	Testes unitários Java de validação, testes de sucesso/falha de sign e validate.
US-03 Gerenciar ciclo de vida do simulador	cmd/simulador, internal/process, internal/release, PortChecker, HealthChecker, ArtifactCache.	Testes de start/status/stop, simulação de porta ocupada, download/cache.
US-04 Provisionar JDK automaticamente	internal/jdk, JDKResolver, SystemJavaLocator, ManagedJavaLocator, AdoptiumDownloader.	Testes com Java presente, Java ausente, cache existente e plataforma suportada.
US-05 Disponibilizar binários multiplataforma	.github/workflows/build.yml, release.yml, checksums, Cosign, version injection.	Execução de CI, build cross-platform, release com tag v*, verificação de checksums.

13.1 Matriz de rastreabilidade por sprint

Sprint	Foco	Elementos de projeto priorizados
Sprint 1	Fundação e entrega contínua.	cmd/assinatura version, cmd/simulador stub, go.mod, estrutura de pastas, build.yml, release.yml.
Sprint 2	Assinatura simulada em modo local.	assinador.jar, validadores, FakeSignatureService, comandos sign/validate, LocalInvoker, JDKResolver.
Sprint 3	Modo servidor e PKCS#11.	SignatureController, HTTPInvoker, ProcessRegistry, HealthChecker,

		Pkcs11Adapter, start/stop/status.
Sprint 4	Simulador e segurança final.	cmd/simulador completo, ArtifactResolver, download de simulador.jar, checksum, Cosign, release final.

14. Decisões de projeto, justificativas, riscos e pendências

14.1 Decisões de projeto

ID	Decisão	Justificativa	Consequência
DP-01	Desenvolver CLIs em Go.	Go facilita gerar binários multiplataforma e possui boa biblioteca padrão para HTTP, processos e arquivos.	Equipe precisa organizar pacotes e lidar com diferenças de SO.
DP-02	Manter assinador.jar em Java 21.	Restrição do projeto e compatibilidade com o domínio Java.	Runner precisa localizar/provisionar Java.
DP-03	Isolar execução Java em internal/invoker.	Evita espalhar chamadas os/exec e HTTP pelo CLI.	Facilita testes com mocks e fallback.
DP-04	Usar ~/.hubsaude/ para cache e estado.	Centraliza arquivos gerenciados pelo Runner.	Exige cuidado com permissões, limpeza e portabilidade.
DP-05	Reutilizar lógica Java entre modo CLI e HTTP.	Evita duplicação de validações e inconsistência de resultados.	Controller e CLI Java devem chamar os mesmos serviços.
DP-06	Publicar artefatos com checksums e assinatura.	Aumenta confiança e integridade da distribuição.	Pipeline fica mais complexo, mas mais seguro.

14.2 Riscos de projeto

Risco	Impacto	Mitigação
Diferenças entre sistemas operacionais para iniciar/encerrar processos.	Falhas em Windows, Linux ou macOS.	Criar camada ProcessManager testável e usar CI nas três plataformas.
Download de JDK/JRE falhar por rede ou URL alterada.	Usuário não consegue executar jars automaticamente.	Permitir --source, cache local, mensagens claras e documentação de instalação manual.
Validações Java e Go divergirem.	Usuário recebe erro inconsistente entre CLI e jar.	Go faz validação inicial mínima; validação final fica no assinador.jar.
Porta 8443 ou porta do assinador ocupada.	Servidor/simulador não inicia.	PortChecker antes de iniciar e flag --port.
Checksum inválido ou artefato corrompido.	Risco de executar arquivo incorreto.	Bloquear execução e limpar arquivo temporário.
PKCS#11 indisponível no ambiente de testes.	Testes de integração com token real podem falhar.	Usar SoftHSM2 ou adaptador simulado.

14.3 Pendências a confirmar com o professor/equipe

☐ Formato final dos parâmetros exigidos pelos casos de uso FHIR de criar e validar assinatura.

☐ Nome exato dos comandos de gerenciamento do assinador.jar: start/status/stop ou subcomando server.

☐ Porta padrão do assinador.jar em modo servidor.

☐ Se o código-fonte do Simulador do HubSaúde deve ser implementado pela equipe ou apenas gerenciado pelo Runner.

☐ Formato oficial do release.json e local definitivo de publicação.

☐ Obrigatoriedade de AppImage/dmg ou aceitação de binários simples gerados por Go.

☐ Política exata de fallback: iniciar servidor automaticamente ou cair para modo local quando servidor não está ativo.

15. Verificação, validação e critérios de aceite do projeto detalhado

15.1 Verificação do projeto

Item	Critério de verificação
Consistência modular	Nenhum pacote internal depende de cmd; dependências seguem direção definida.
Interfaces	Comandos, endpoints, DTOs e arquivos locais estão descritos com entradas e saídas.
Rastreabilidade	Cada requisito US-01 a US-05 possui elementos de projeto e testes associados.
Tratamento de erros	Todas as categorias principais de erro possuem código e comportamento definido.
Portabilidade	Há previsão explícita para Windows, Linux e macOS.
Segurança de artefatos	Checksums e assinatura de release estão integrados ao projeto.

15.2 Critérios de aceite de implementação derivados do projeto

☐ O comando assinatura version imprime dev localmente e a tag do release quando compilado pela pipeline.

☐ go build ./..., go vet ./... e go test ./... passam localmente e no CI.

☐ O CLI assinatura executa sign e validate em modo local chamando assinador.jar.

☐ O assinador.jar rejeita parâmetros inválidos antes de simular qualquer operação.

☐ O assinador.jar expõe /health, /sign e /validate em modo servidor.

☐ O CLI reutiliza servidor saudável e não cria instâncias duplicadas desnecessariamente.

☐ O CLI simulador inicia, consulta e encerra o Simulador do HubSaúde.

☐ O JDK/JRE é localizado ou baixado automaticamente em ambiente sem Java compatível.

☐ Os binários são gerados para Windows, Linux e macOS e publicados com checksum.

☐ Os artefatos de release são assinados quando a etapa Cosign estiver habilitada.

16. Referências

- IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
- ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise — Architecture description.
- Arquivo especificacao.md — Sistema Runner - Trabalho Prático.
- Arquivo design.md — Sistema Runner - Design, com diagramas C4.
- Arquivo plano-revisitado-v2.md — Plano Revisado #2, sprints e histórias derivadas.
- Arquivo sprint-1-tasks.md — Sprint 1: tarefas operacionais e decisões técnicas.
- Documentação dos casos de uso de segurança FHIR indicados no enunciado do trabalho.

Apêndice A — Checklist rápido para implementação

☐ Criar go.mod com módulo github.com/kyriosdata/runner.

☐ Criar cmd/assinatura/main.go com Cobra e comando version.

☐ Criar cmd/simulador/main.go com stub inicial e evolução para start/stop/status.

☐ Criar internal/cli, internal/invoker, internal/jdk, internal/release, internal/process e internal/output.

☐ Criar projeto Maven do assinador.jar em Java 21.

☐ Implementar DTOs e validadores Java antes do serviço fake.

☐ Implementar LocalInvoker antes do HTTPInvoker.

☐ Adicionar workflows build.yml e release.yml.

☐ Gerar checksums e, quando possível, assinaturas Cosign.

☐ Documentar comandos de uso e mensagens de erro.

Apêndice B — Exemplo de sequência textual completa

```
Usuário
  │
    ├─ assinatura sign --input doc.xml --signer-id 123
--cert-alias token-a1
  │
CLI assinatura
  │  valida flags básicas
```

```
  |  consulta ProcessRegistry
  |  escolhe HTTPInvoker ou LocalInvoker
  |
assinador.jar
  |  valida SignRequest
  |  consulta Pkcs11Adapter quando solicitado
  |  FakeSignatureService gera resposta simulada
  |
CLI assinatura
  |  formata SignResponse
  ▼
Usuário recebe assinatura simulada ou erro orientativo
```